R Bootcamp 3 - Data Visualization

Giovanni Castro Irizarry
UCLA M183/M136C

2025-05-01

Acknowledgments

Much of this lesson plan was adapted from Babraham Bioinformatics' *Introduction to* `ggplot2` manual online. A direct link to the article is provided: https://www.bioinformatics.babraham.ac.uk/training/ggplot_course/Introduction%20to%20ggplot.pdf

Introduction

Today, we are going to focus on data visualization techniques. In research papers, most data visualization appears as graphs and/or tables. Graphs are typically preferred as they clearly demonstrate the point you are making with the data, and tables often are known as Big Ugly Tables of Numbers (BUTON) because they are far less inviting to the reader than a graph.

A quick intoduction to `ggplot`

What is `ggplot`? How does it work?

As such, I will focus primarily today on the `ggplot2` package because it is the best at creating beautiful graphs. The package `ggplot2` is automatically wrapped up in the `tidyverse` package, so we only need to load in the `tidyverse`.

`ggplot` is the package we are going to use to make graphical representations of our findings. This week, I am going to demonstrate some of the basics of how it works.

`ggplot` is essentially and additive drawing tool for visualizations of data. By additive, I mean that each visual feature you add to the plot is done with a "+" operator: you are quite literally adding elements to the visual in order to build it. R knows how to process these visual elements through the graph's aesthetics property created using the aes() function. This function tells R things like what data belongs on what axis, what color to mark data points, what size to make data points, and other basics of how to translate a dataset into a visualization.

To create a graph in `ggplot2`, you must include several parameters in its construction: `aes()` and `geom()`. `aes()` is used to load in the data and specify the x- and y-axes. The `geom()` parameter helps us decide which style of graph `ggplot2` will construct.

The following graphs can automatically be constructed with `geom()`:

- `geom_bar()` The Bar Graph
- `geom_point()` Scatterplot
- `geom_line()` Add a line to the graph
- `geom_smooth()` Add a line to the graph (typically a linear regression line)
- `geom_histogram()` The histogram

- `geom_boxplot()` The boxplot
- `geom_hline()` or `geom_vline()` Add a horizontal or vertical line

The general form of your code will look like the following:

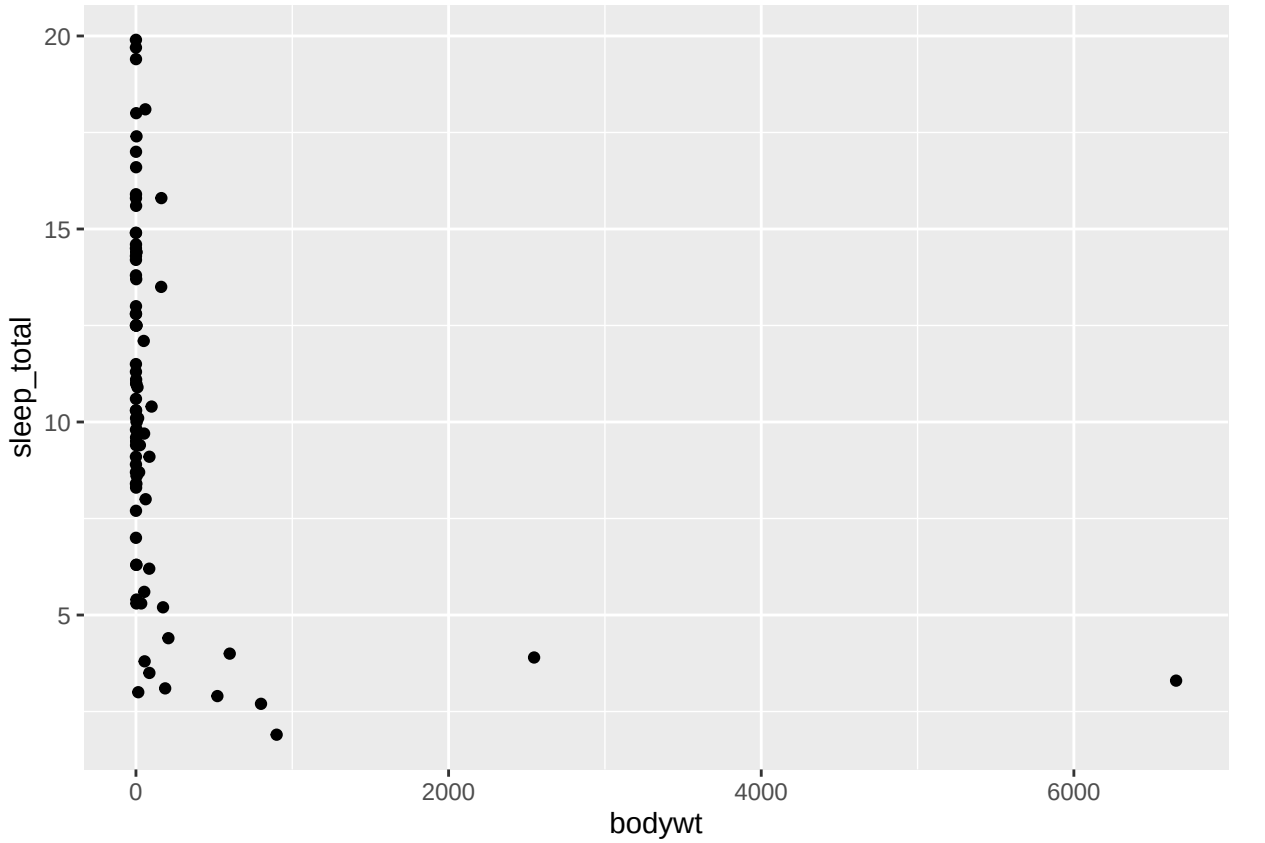
```
mygraph <- ggplot(mydata, aes(variable for x axis, variable for y axis)) + geom_GRAPHTYPE()
```

Scatterplot

Let's actually start graphing! We are going to first graph a scatterplot with the `msleep` dataset (which is already loaded into `R`!)

Let's look at the relationship between numbers of hours slept per day (`sleep_total`) and the weight of the animal (`bodywt`).

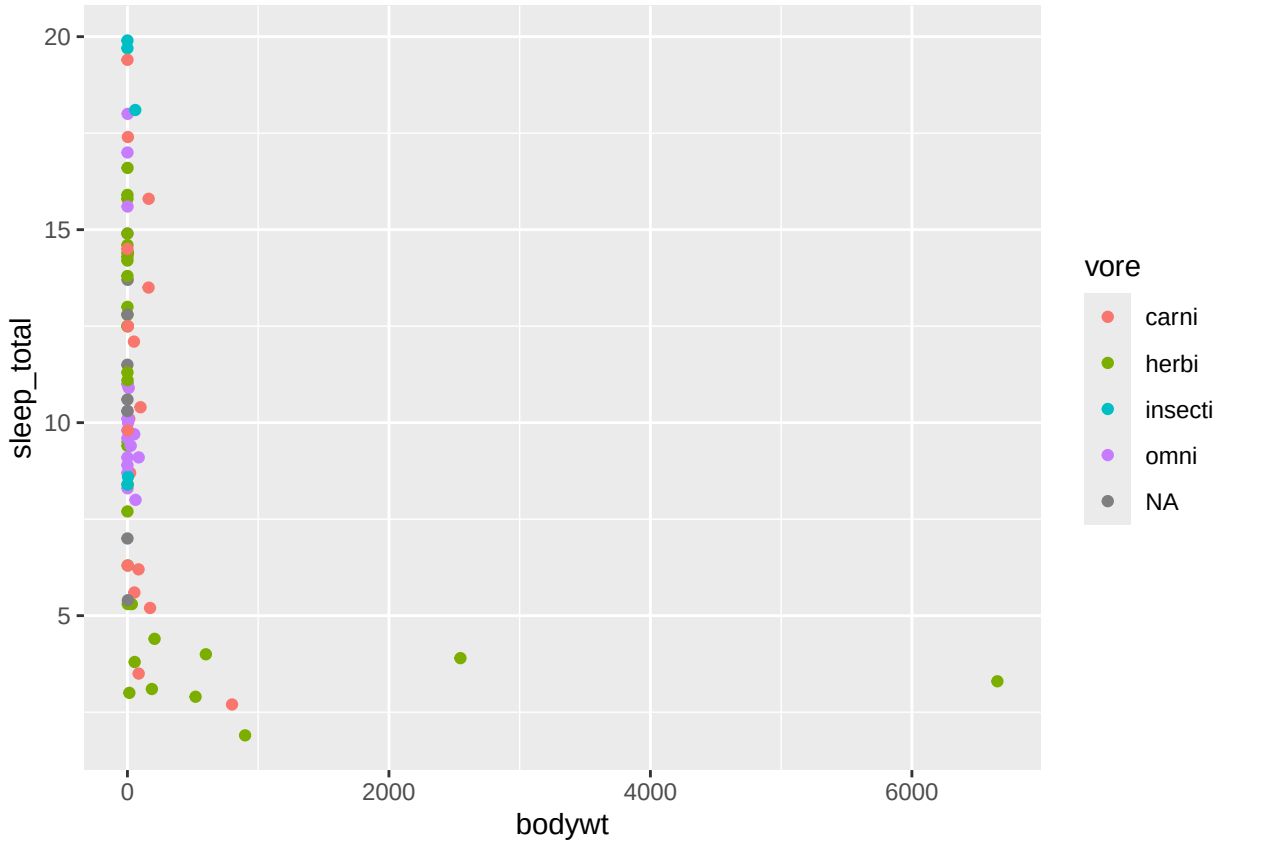
```
scatterplot <- ggplot(data = msleep,
                      aes(x = bodywt,
                          y = sleep_total)) +
  geom_point()

# scatterplot <- ggplot(data = msleep, aes(x = bodywt, y = sleep_total)) + geom_point()

scatterplot
```



Now we can change the colors of the points based on what trophic level each animal is. You set the "col" parameter:

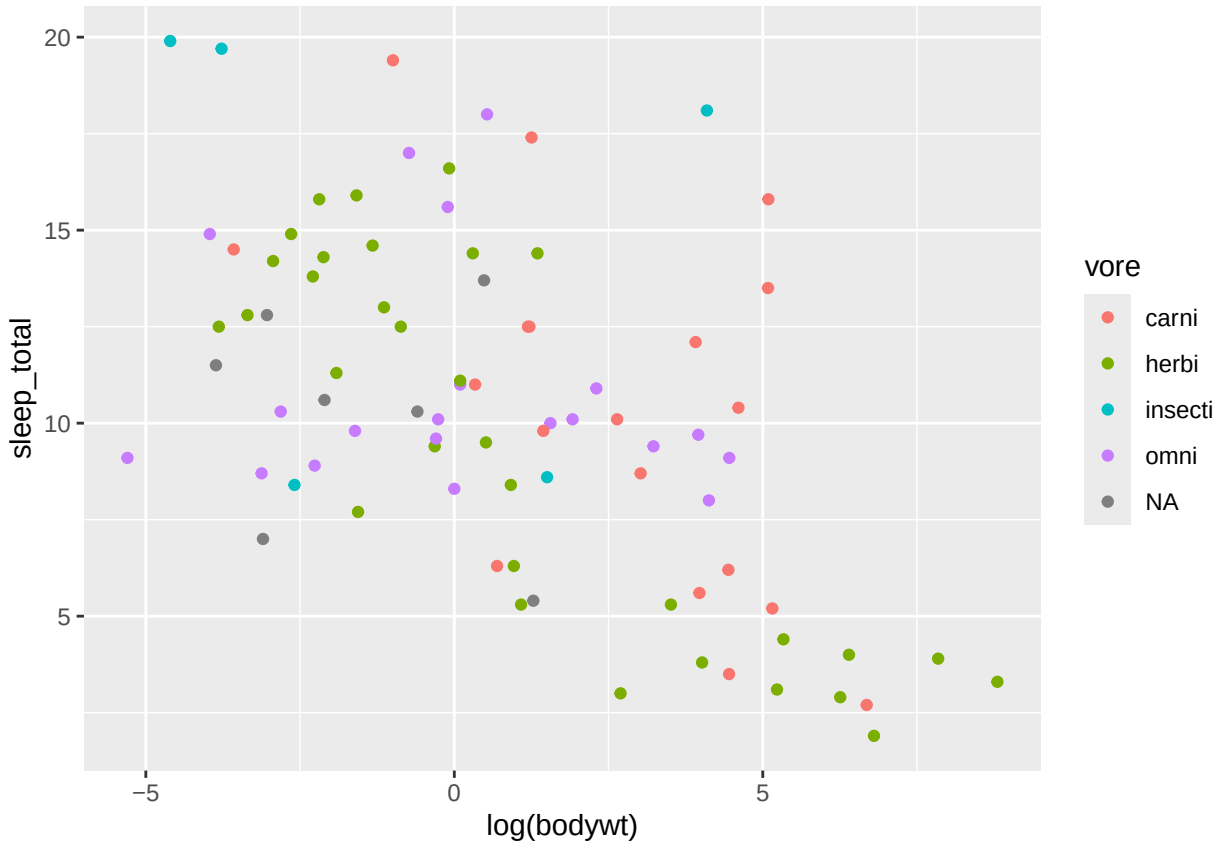
```
scatterplot <- ggplot(data = msleep,
                      aes(x = bodywt,
```

```
                        y = sleep_total,
                        col = vore)) +
  geom_point()

scatterplot
```



Now, we know the relationship is not linear. However, we still want to look at variation within the species by bodyweight. We should take the log of the `bodywt` variable.

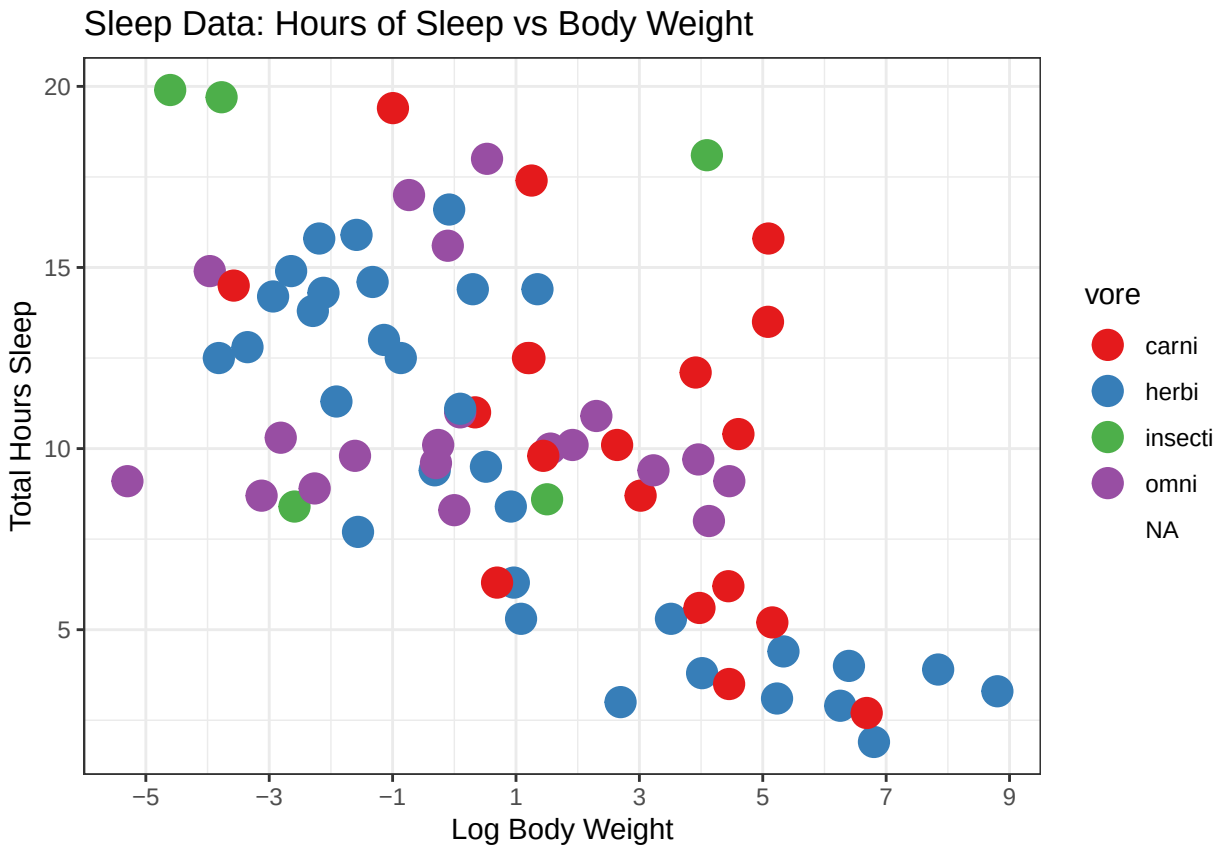
```
scatterplot <- ggplot(data = msleep,
                      aes(x = log(bodywt),
                        y = sleep_total,
                        col = vore)) +
  geom_point()

scatterplot
```



Lastly, let's perform some cosmetic commands to make the graph look as good as possible:

```
scatterplot <- scatterplot +
  geom_point(size = 5) +
  xlab("Log Body Weight") +
  ylab("Total Hours Sleep") +
  ggtitle("Sleep Data: Hours of Sleep vs Body Weight") +
  scale_colour_brewer(palette = "Set1") +
  scale_x_continuous(breaks = seq(-5, 10, 2)) +
  theme_bw()

scatterplot
```

```
## Warning: Removed 7 rows containing missing values or values outside the scale range
## (`geom_point()`).
## Removed 7 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Here are some other color palettes to be used with commands like `scale_colour_brewer(palette = "Set1")`

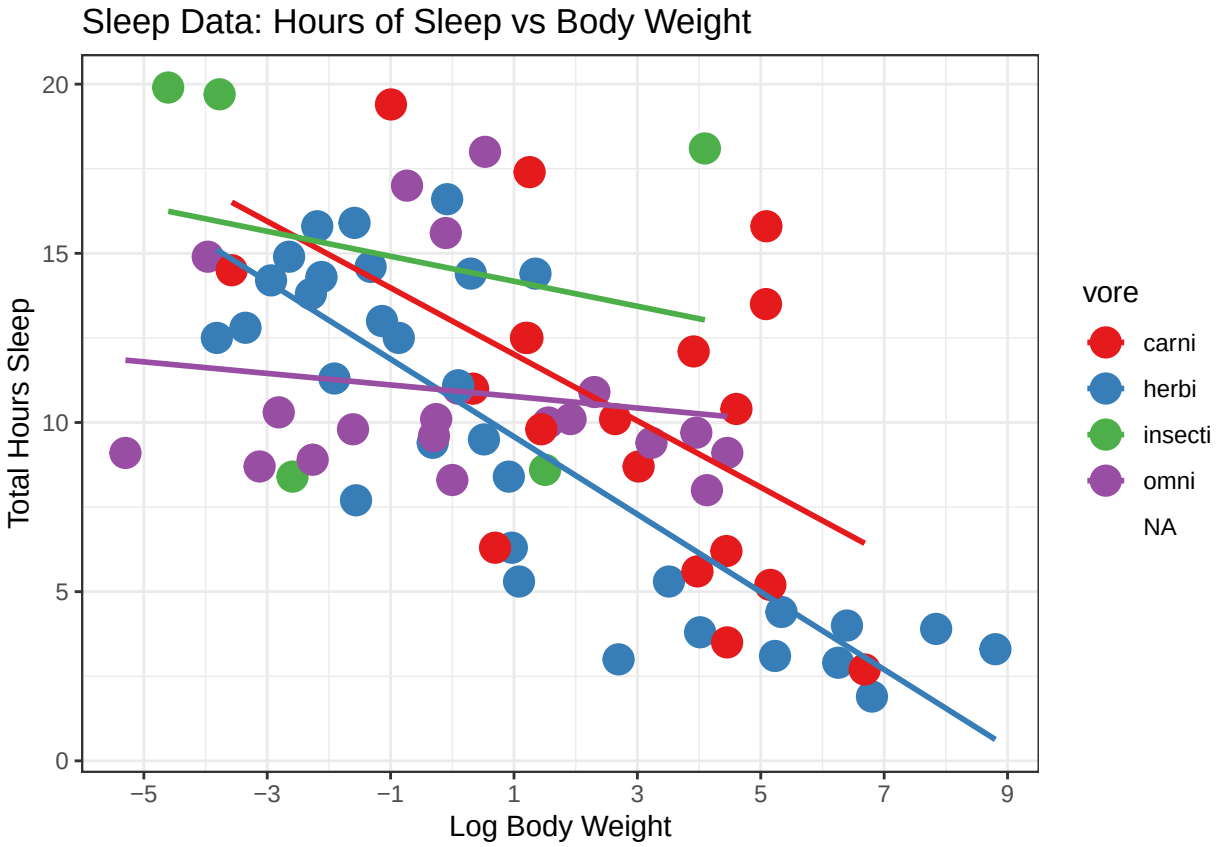
```
library(RColorBrewer)
display.brewer.all(type = "qual")
```

Next, let's draw some linear regression lines. The `geom_smooth()` command here draws a linear regression line for each category separated. We will talk about linear regression next week:

```
scatter_lm <- scatterplot +
  geom_smooth(method = 'lm',
              se = FALSE)

scatter_lm
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 7 rows containing missing values or values outside the scale range
## (`geom_point()`).
## Removed 7 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Feel free to go through and run each individual + to understand what each portion of the plot is doing. You can also play around with the graphs to make them to your liking. I am not expecting beautiful and perfect graphs, but at the very least, your groups should be outputting simplistic and accurate graphs (and make sure to change the x-axis, y-axis, and graph title names).

Let's move onto another dataset. This time, we will be using the `mtcars` dataset. Try this example on your own!

Try to create a nice-looking scatterplot with Horsepower (`hp`) on the x-axis, Quarter Mile Time (`qsec`) on the y-axis, and a linear regression line illustrating the correlation.

```
#Work in groups. Type your code here
```

A graph such as this one above would be perfectly adequate in your final paper.

Histograms

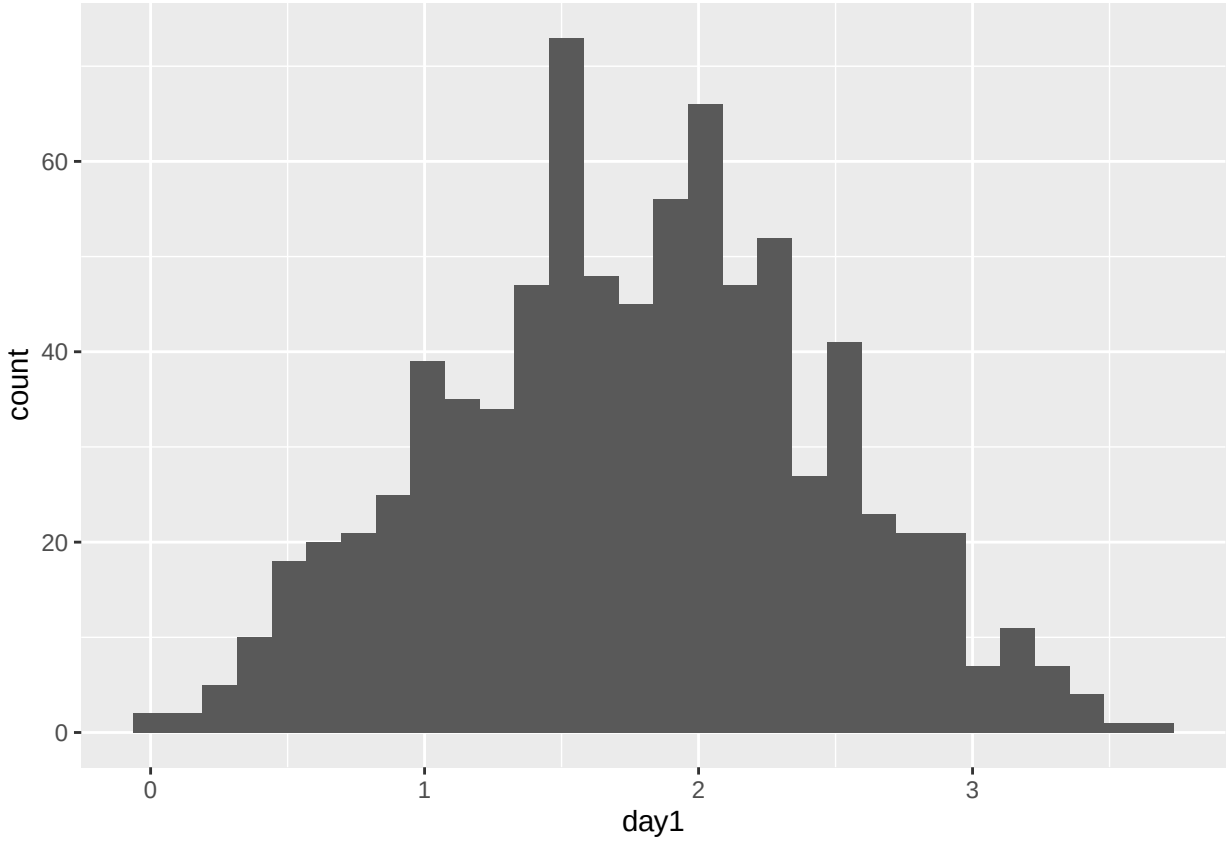
First, we need to create our dataframe. I downloaded this .txt file from the internet, and now we need to do some coding gymnastics to make the data ready to be coded.

```
festival_data <- read.csv(url("https://www.dropbox.com/s/73gr4v0llal20ro/festival.data.csv?dl=1")) %>%
  select(-1)
```

Now that we are done cleaning up the data, let's create a histogram of Day 1 Hygiene:

```
day1_histogram <- ggplot(festival_data,
                         aes(x = day1)) +
  geom_histogram()

day1_histogram
```

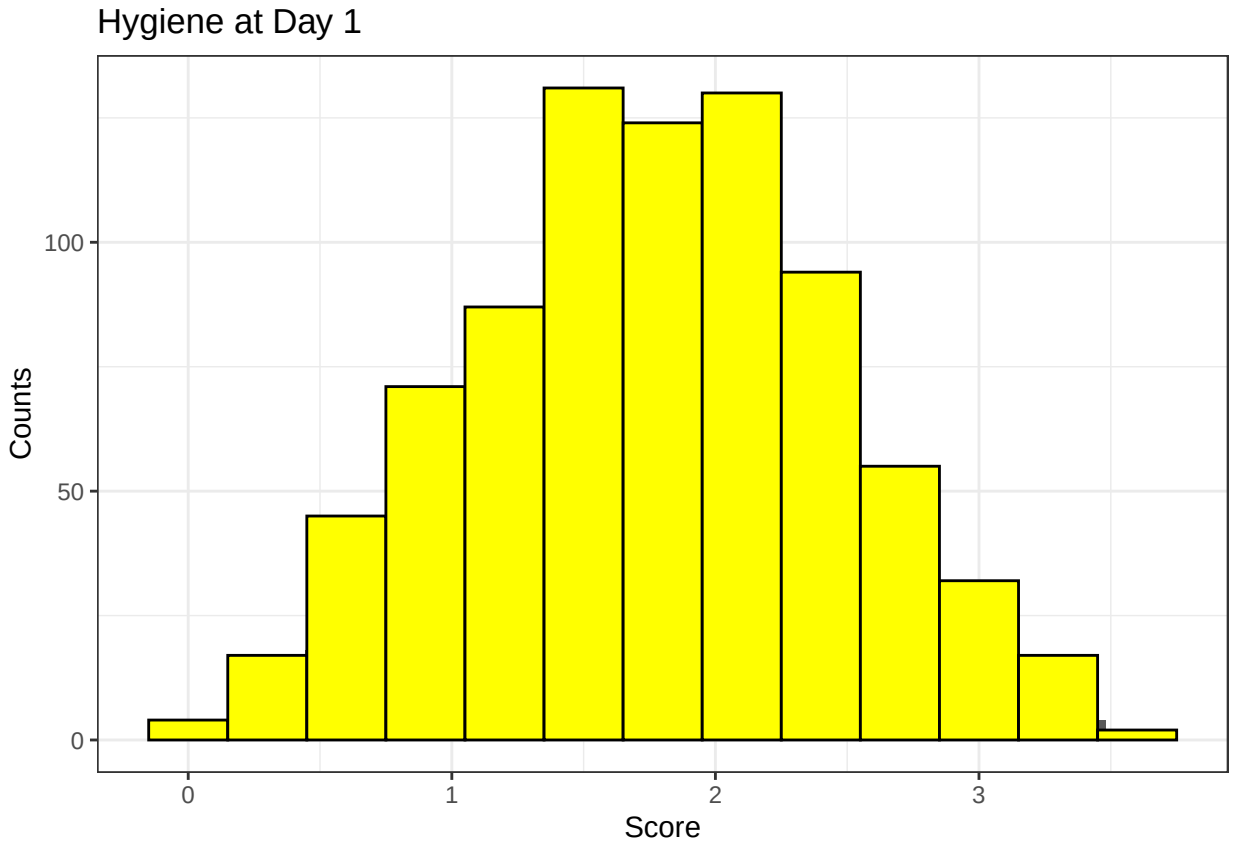
```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



This looks pretty dreary. Let's clean it up by changing the Bin Width (the size of each bin), color, labels, and title of the graph:

```
day1_histogram <- day1_histogram +
  geom_histogram(binwidth = 0.3,
                 color = "black",
                 fill="yellow") +
  labs(x = "Score",
       y = "Counts") +
  ggtitle("Hygiene at Day 1") +
  theme_bw()

day1_histogram
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

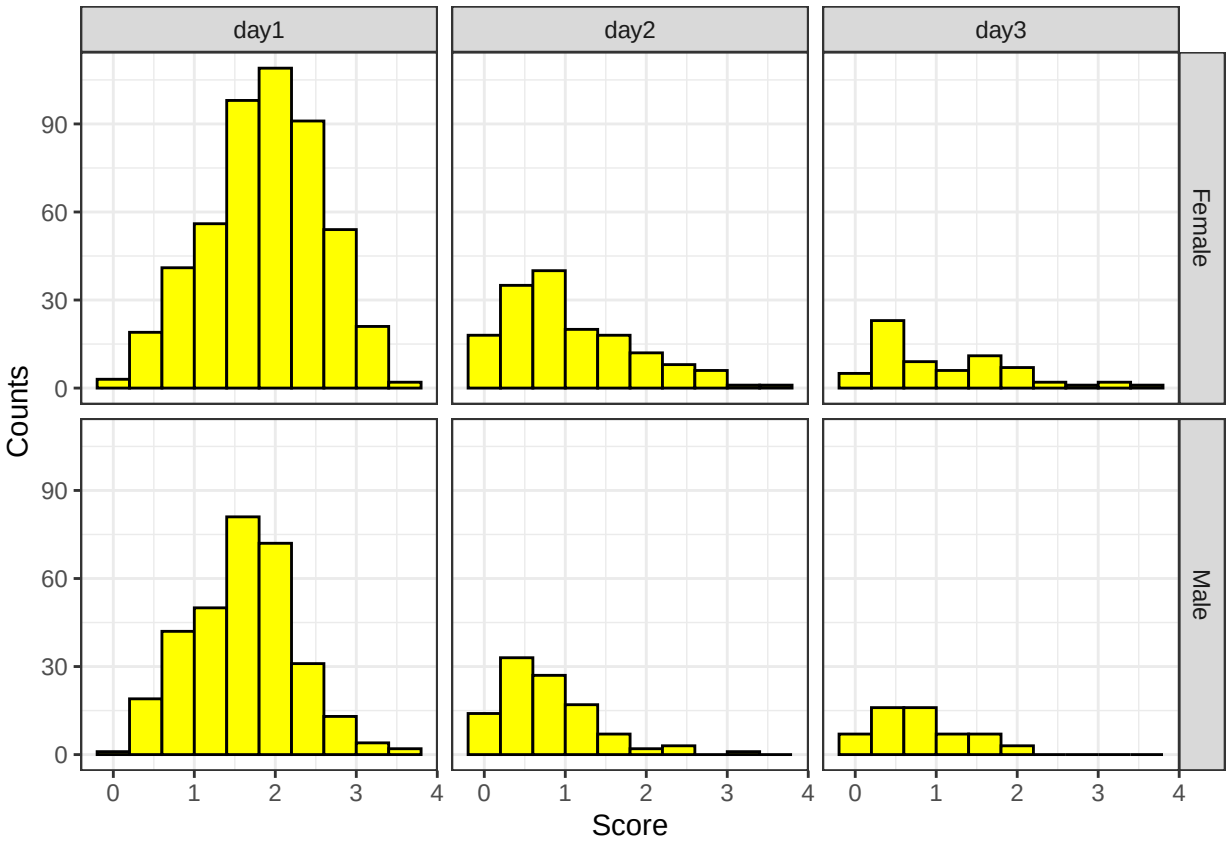


Much better.

I doubt we will need to do much more trickery with the histogram plots, but just in case, here is some code that will allow you to put multiple plots on one page:

```
library(reshape2)

festival_data_stack <- melt(festival_data,
                            id = c("ticknumb","gender"))

colnames(festival_data_stack)[3:4] <- c("day", "score")

histogram_3days <- ggplot(festival_data_stack,
                          aes(score)) +
  geom_histogram(binwidth = 0.4,
                 color = "black",
                 fill = "yellow") +
  labs(x = "Score", y = "Counts") +
  facet_grid(gender ~ day) +
  theme_bw()

histogram_3days
```

```
## Warning: Removed 1232 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



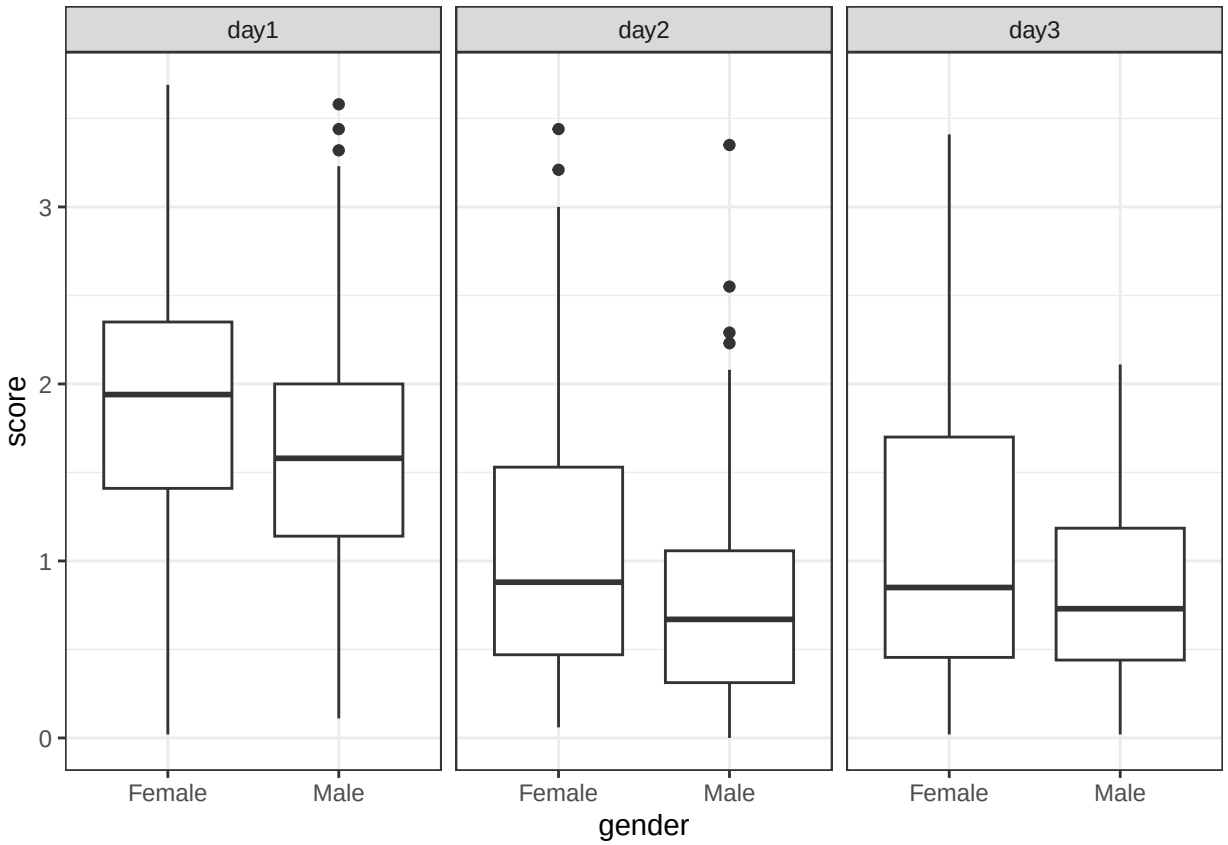
This fancier plot lets you compare across gender and days to see how the hygiene compared across covariates.

Boxplots

Let's draw boxplots with this data.

```
boxplot <- ggplot(festival_data_stack,
                  aes(x = gender,
                      y = score)) +
  geom_boxplot() +
  facet_grid(~day) +
  theme_bw()

boxplot
```

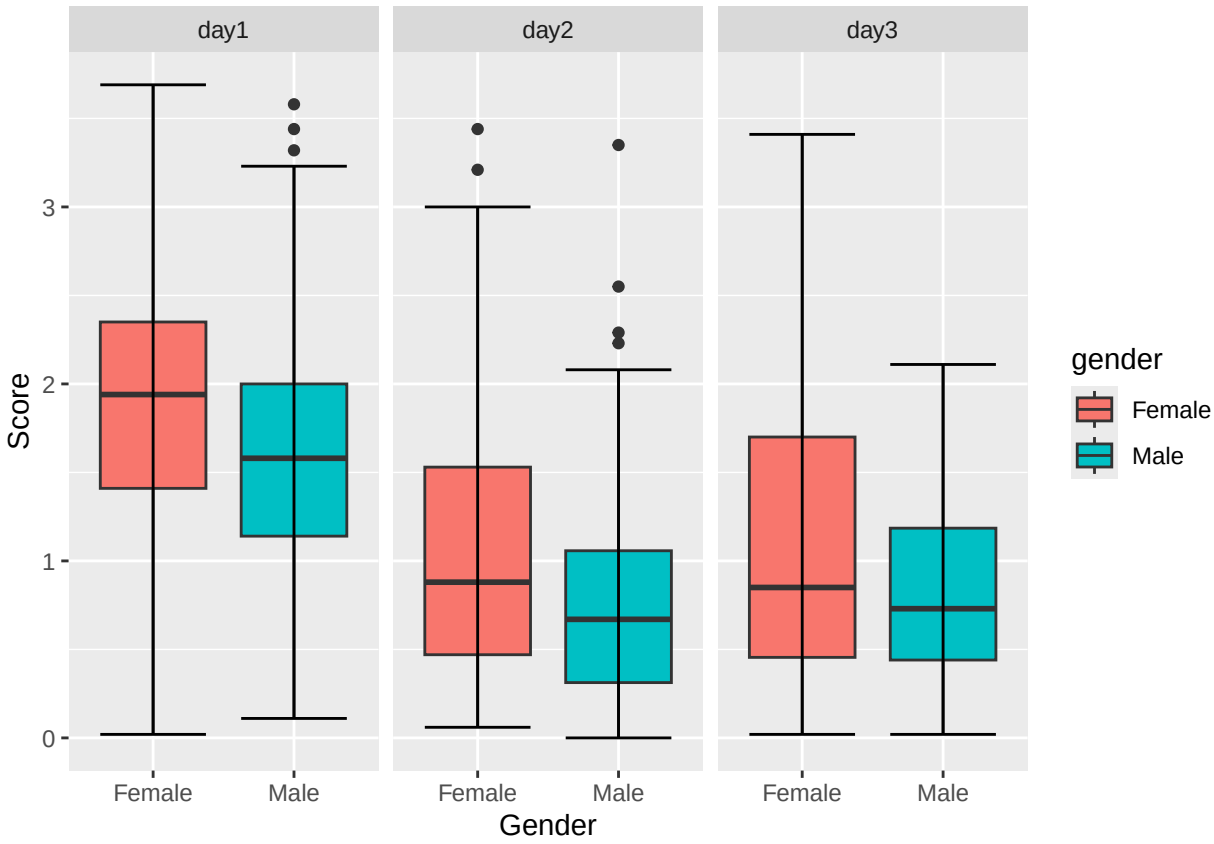
```
## Warning: Removed 1232 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Pretty good, but if you want to add color or change the descriptive variable names:

```
boxplot <- ggplot(festival_data_stack,
                  aes(x = gender,
                      y = score,
                      fill = gender)) +
  geom_boxplot() +
  stat_boxplot(geom = "errorbar") +
  facet_grid(~day) +
  xlab("Gender") +
  ylab("Score")

boxplot
```

```
## Warning: Removed 1232 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
## Removed 1232 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Bar Charts

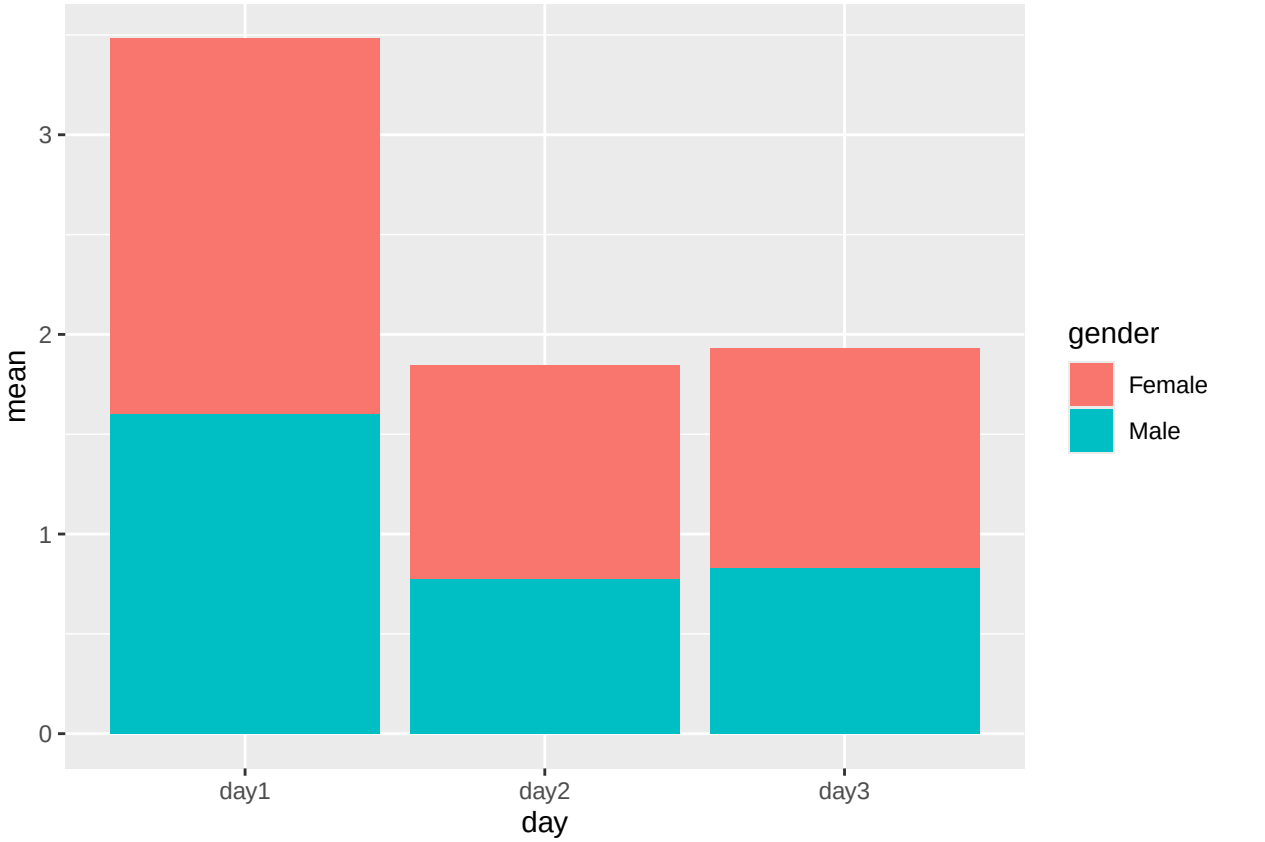
Last, but certainly not least, is the Bar Charts. I do not feel I need to explain much in the coding here as we have made clear that simplistic graphs are for the best. However, I would feel I wronged you all if I did not provide the code that can transform these simplistic bar charts.

```
festival_data_stack2 <- na.omit(festival_data_stack)

score_sem <- ddply(festival_data_stack2,
                   c("gender", "day"),
                   summarise,
                   mean = mean(score),
                   sem = sd(score)/sqrt(length(score)))

score_sem
```

```
##   gender  day      mean        sem
## 1 Female day1 1.8784413 0.03170343
## 2 Female day2 1.0743396 0.06055344
## 3 Female day3 1.0997015 0.09895861
## 4   Male day1 1.6020635 0.03619580
## 5   Male day2 0.7732692 0.05847218
## 6   Male day3 0.8291071 0.07209944
```

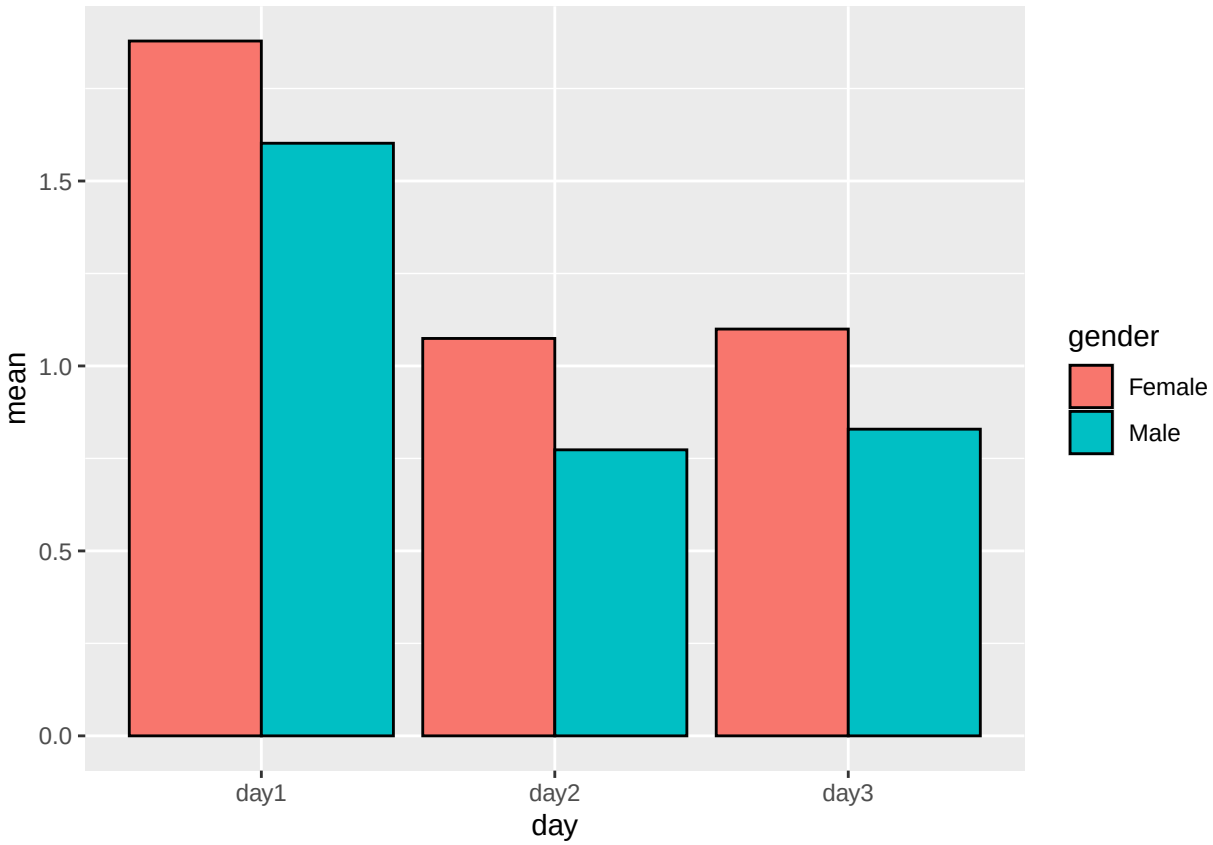
```
festival_bar <- ggplot(score_sem,
                       aes(day, mean,
                           fill = gender)) +
```

```
  geom_bar(stat = "identity")

festival_bar
```



Not great. It would be much better if we could compare the heights of bars directly. You would do so like this:

```
festival_bar <- ggplot(score_sem,
                       aes(x = day,
                           y = mean,
                           fill = gender)) +
  geom_col(position = "dodge",
           colour = "black")

festival_bar
```

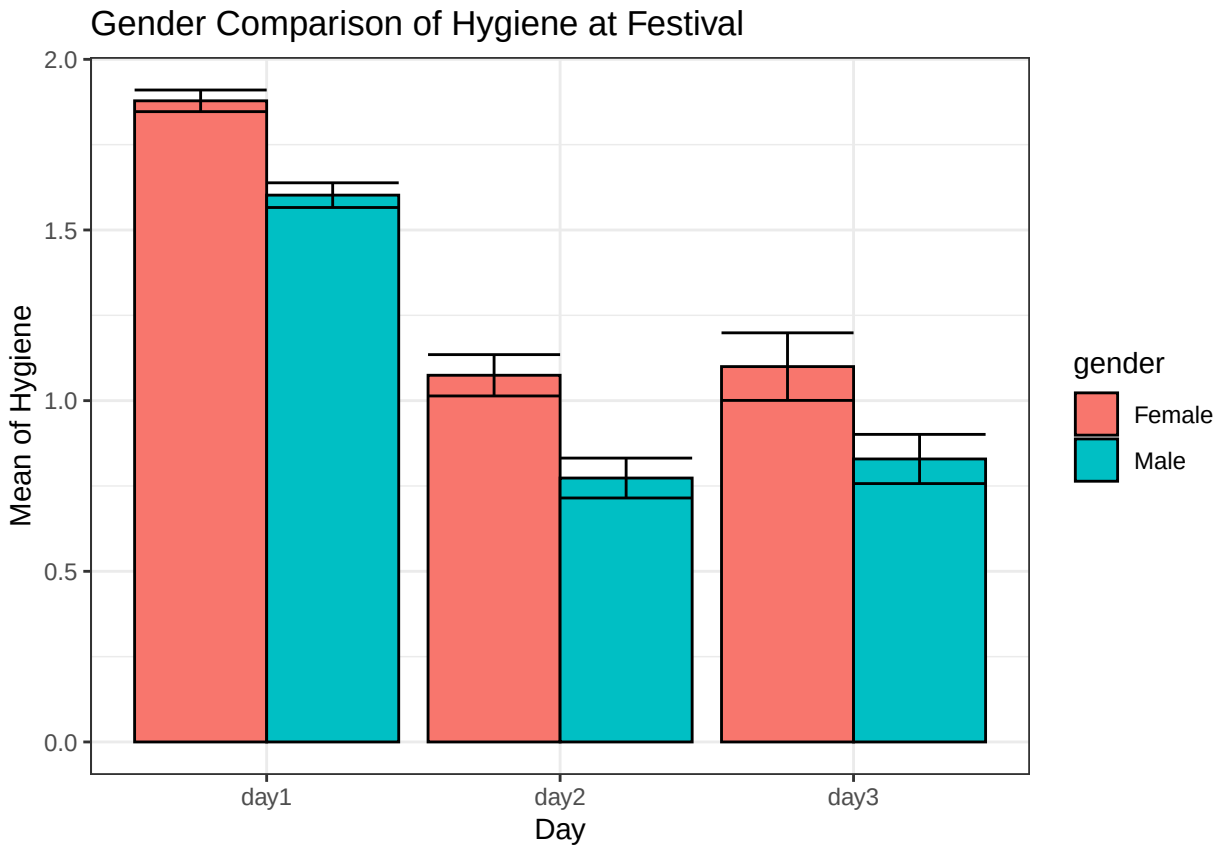


Last, let's add some error terms:

```
festival_bar <- ggplot(score_sem,
                       aes(x = day,
                           y = mean,
                           fill = gender)) +
  geom_col(position = "dodge",
           colour = "black",
           stat = "identity") +
  geom_errorbar(aes(ymin = mean - sem,
                    ymax = mean + sem),
                position = "dodge") +
  xlab("Day") +
  ylab("Mean of Hygiene") +
  ggtitle("Gender Comparison of Hygiene at Festival") +
  theme_bw()
```

```
## Warning in geom_col(position = "dodge", colour = "black", stat = "identity"):
## Ignoring unknown parameters: `stat`
```

```
festival_bar
```



Save a Graph

To save a graph as a .png file (THIS IS IMPORTANT SINCE MOST OF YOU WILL WRITE YOUR FINAL PAPER IN A WORD DOCUMENT), use the `ggsave` function. Simply put the plot's object name into the `ggsave` function, and it will save the .png file to the file destination on your laptop that the R Markdown file is being saved to. You can see an example below:

```
scatterplot_saved <- ggsave(scatterplot, file = "scatterplot.png")

scatterplot2_saved <- ggsave(scatterplot2, file = "scatterplot2.png")
```

Stargazer

Now, let's make a Big Ugly Table of Numbers (BUTON) by using the `stargazer` package, which will automatically tell us if our data is statistically significant to the `p < 0.05`.

The first thing you need to do to make the `stargazer` package work is download LaTex. This will help you knit your documents to PDF and output pretty tables and graphs. Please go to the website https://www.latex-project.org/get/ to download your LaTex file for your computer.

The `stargazer` package is tricky first of all because you *must* put `results = 'asis'` in the `{r}` area at the top of the code chunk. Additionally, you must also knit the document into PDF in order to output a `stargazer` table.

As an example:

```
stargazer(attitude,
          type = 'latex',
          title = "Summary Statistics",
          no.space = TRUE)
```

% Table created by stargazer v.5.2.3 by Marek Hlavac, Social Policy Institute. E-mail: marek.hlavac at gmail.com % Date and time: Thu, May 01, 2025 - 02:33:50

Table 1: Summary Statistics

Statistic	N	Mean	St. Dev.	Min	Max
rating	30	64.633	12.173	40	85
complaints	30	66.600	13.315	37	90
privileges	30	53.133	12.235	30	83
learning	30	56.367	11.737	34	75
raises	30	64.633	10.397	43	88
critical	30	74.767	9.895	49	92
advance	30	42.933	10.289	25	72

This first stargazer table just outputs summary statistics. Let's output a stargazer table of a linear regression model:

```
lm_function <- lm(hp ~ qsec + mpg + cyl + drat + wt,
                  data = mtcars)

stargazer(lm_function,
          type = 'latex',
          title = "OLS Results",
          omit.stat = c("LL", "ser", "f"),
          no.space = TRUE)
```

% Table created by stargazer v.5.2.3 by Marek Hlavac, Social Policy Institute. E-mail: marek.hlavac at gmail.com % Date and time: Thu, May 01, 2025 - 02:33:50

```
regression <- stargazer(lm_function,
                        type = 'latex',
                        title = "OLS Results",
                        omit.stat = c("LL", "ser", "f"),
                        no.space = TRUE)
```

% Table created by stargazer v.5.2.3 by Marek Hlavac, Social Policy Institute. E-mail: marek.hlavac at gmail.com % Date and time: Thu, May 01, 2025 - 02:33:50

Table 2: OLS Results

	Dependent variable:
	hp
qsec	-16.180^{***}
	(5.690)
mpg	-2.406
	(2.406)
cyl	10.462
	(9.319)
drat	15.280
	(18.409)
wt	19.145
	(14.784)
Constant	302.529
	(185.031)
Observations	32
R^2	0.816
Adjusted R^2	0.781
Note:	*p<0.1; **p<0.05; ***p<0.01

Table 3: OLS Results

	Dependent variable:
	hp
qsec	-16.180^{***}
	(5.690)
mpg	-2.406
	(2.406)
cyl	10.462
	(9.319)
drat	15.280
	(18.409)
wt	19.145
	(14.784)
Constant	302.529
	(185.031)
Observations	32
R^2	0.816
Adjusted R^2	0.781
Note:	*p<0.1; **p<0.05; ***p<0.01